MP2.2 : Highly Available and Scalable Tiny SNS

Phani Jyothi Kurada
UIN: 335006950
CSCE 662

November 25, 2025

1 Introduction

This project implements a distributed social-network-style service (SNS) spanning **three clusters**, each containing a **master** and **slave** TSD server pair, supported by cross-cluster propagation using **Synchronizers**, and a **Coordinator** for discovery and heartbeat management. The client program (`tsc`) communicates with TSD servers over gRPC, while Synchronizers exchange state via **RabbitMQ**.

The core goals are:

- Provide a global namespace of users across all clusters.
- Ensure FOLLOW/UNFOLLOW operations propagate cluster-wide.
- Maintain persistent user, follower, and timeline data across restarts.
- Provide real-time TIMELINE streaming using server-side watchers.
- Achieve eventual cross-cluster consistency using periodic synchronization.

The system guarantees:

- **Global visibility**: LIST returns all users registered across all clusters.
- **Cross-cluster follows**: FOLLOW writes follower state to all clusters.
- **Consistent timelines**: Posts immediately fan out to local and remote followers.
- **Persistence**: Disk files store user lists, follower lists, follow lists, and timelines.

2 Environment Setup

2.1 Prerequisites

- Ubuntu-based Docker container for running all components.
- Tools: `g++`, `make`, `protobuf`, `grpc`, `jsoncpp`.
- `docker-compose` for managing RabbitMQ and the MP2.2 environment.
- Libraries:
  - **glog** for structured logging.
  - **POSIX semaphores** for file-level concurrency.
  - **RabbitMQ C client** (`librabbitmq4`) for synchronizer messaging.

- RabbitMQ setup as described in the project README:
  - Enable the `rabbitmq_management` plugin.
  - Expose ports `5672` (AMQP) and `15672` (management UI).
  - Ensure container names match the synchronizer configuration.

2.2 Build and Run

Docker Environment Setup

```
docker-compose up -d
docker exec -u root csce438_mp2_2_container bash -lc \
   "apt-get update && apt-get install -y librabbitmq4"
```

Launching the MP2.2 Stack

```
docker exec -it csce438_mp2_2_container bash
cd /home/csce438/mp2_2
./startup.sh
```

Building Binaries

```
make -f Makefile.txt
```

Starting Coordinator and TSD Servers

The `startup.sh` script automatically launches: - Coordinator - Six TSD servers (3 clusters $\times$ master/slave) - Six Synchronizers

Client Usage

To launch clients for different users:

```
./tsc -h localhost -k 9000 -u 1
./tsc -h localhost -k 9000 -u 2
./tsc -h localhost -k 9000 -u 3
./tsc -h localhost -k 9000 -u 4
./tsc -h localhost -k 9000 -u 5
```

Users can then issue commands such as `LIST`, `FOLLOW`, `UNFOLLOW`, `POST`, and `TIMELINE`.

3 System Components

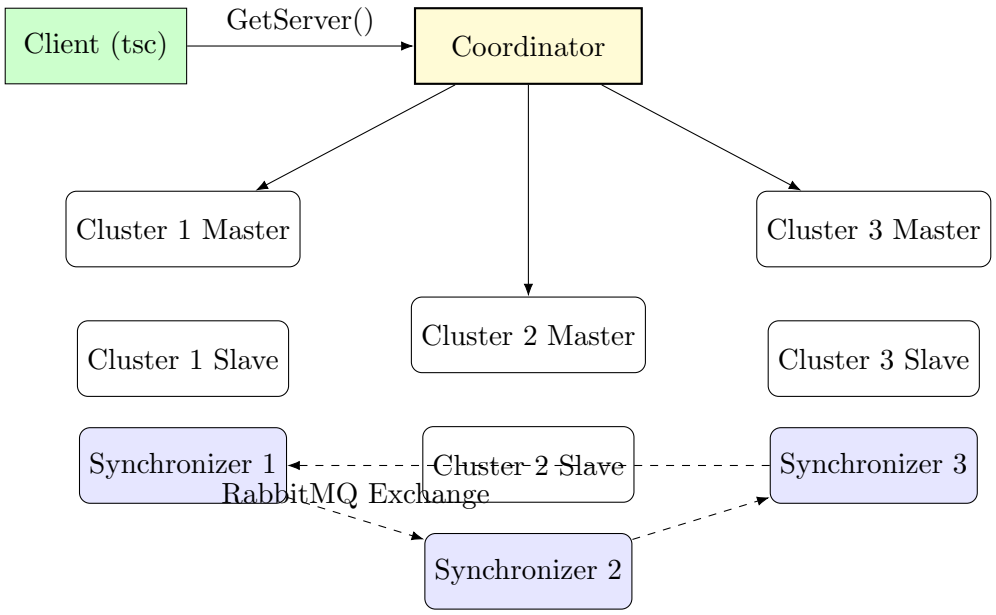


Figure 1: Overall MP2.2 Distributed SNS Architecture

3.1 Coordinator

The Coordinator registers TSD servers and Synchronizers, assigns clients to server pairs based on client ID, and handles periodic heartbeat checks.

Responsibilities:

- Maintain mapping: `clusterID → (master, slave)`.
- Provide client-directed server discovery via `GetServer`.
- Provide global follower visibility via `GetAllFollowerServers`.
- Track liveness of servers and synchronizers.

3.2 TSD Servers

Each TSD instance implements SNS functionality: LOGIN, LIST, FOLLOW, UNFOLLOW, and TIMELINE. Queries interact with a file-based persistent store organized as:

```
cluster_<cid>/<sid>/
    all_users.txt
    <uid>_followers.txt
    <uid>_follow_list.txt
    <uid>_timeline.txt
```

Responsibilities:

- Manage user login (create files if missing; avoid duplicates).

- Maintain global LIST view (merged across clusters).
- On FOLLOW:
  - Validate target exists globally.
  - Append follower entries to all clusters.
  - Append follow-list for the requester.
- On UNFOLLOW: remove follower entries across clusters.
- On POST:
  - Store `poster`message—.
  - Immediately append to all followers' timeline files (all clusters).
- Real-time streaming:
  - Return latest 20 posts (excluding self).
  - Spawn watcher thread to forward new timeline entries.

3.3 Synchronizer

Each cluster has a Synchronizer that periodically merges state across clusters.

Responsibilities:

- Merge `all_users.txt` across clusters.
- Propagate FOLLOW/UNFOLLOW operations.
- Fan out timeline updates across clusters via RabbitMQ.

Message Flow:

- Publish timeline updates with poster ID and message body.
- Consume updates from other synchronizers.
- Append missing messages to follower timelines.

Synchronization interval: `sleep(5)` seconds.

3.4 Client (tsc)

The client discovers its assigned TSD via the Coordinator, then issues commands.

Responsibilities:

- LIST: Display global user list.
- FOLLOW/UNFOLLOW: Update across clusters.
- TIMELINE: Display last 20 posts and stream new ones.
- POST: Publish new messages.

The client ensures:

- Poster ID is displayed for each post.
- Self-posts are not duplicated in the stream.

4 Timeline Semantics

- Posts stored as: `poster`message—.
- On POST:
  - Local TSD writes to all follower timeline files (every cluster).
  - Synchronizer forwards cross-cluster updates.
- New followers **do not receive backfilled history**.
- Old posts remain intact and are not truncated.

5 Test Cases

5.1 Test Case 1: Basic Login and Global LIST Consistency

Objective: Validate that user login creates persistent user entries and that all TSD servers return a globally consistent union of users across clusters.

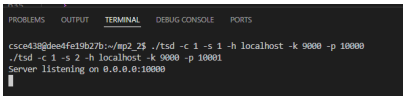
Procedure:

1. Start the full system using `startup.sh`.
2. Launch clients for Users 1 and 4 from different clusters.
3. Issue the `LIST` command on each client followed BY Following each other and posting messages on TIMELINE

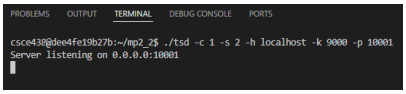
Expected Result: All clients display the global set of users {1, 4} regardless of cluster. Follower lists remain empty. User files are created across clusters without duplication. Followers list gets updated after following each other and exchange of messages posted can be seen on TIMELINE of each user.

Screenshots for Test Case 1

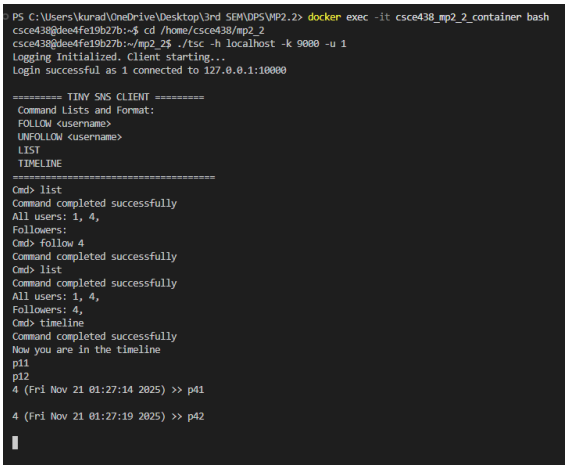
Starting Coordinator



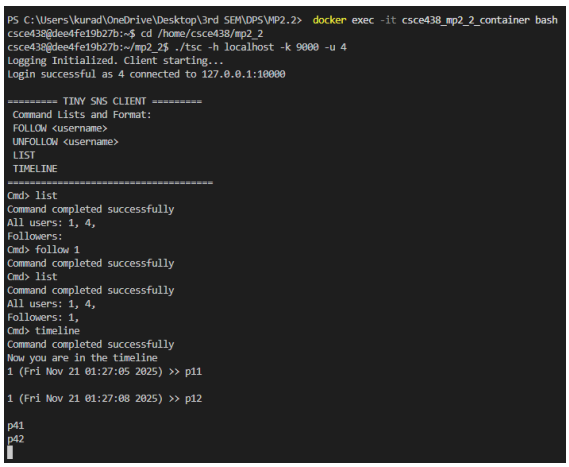
Starting Server 1



Starting Server 2



Creating User 1



Creating User 4

Figure 2: Test Case 1 Execution Screenshots

5.2 Test Case 2: Cross-Cluster FOLLOW and Timeline Propagation

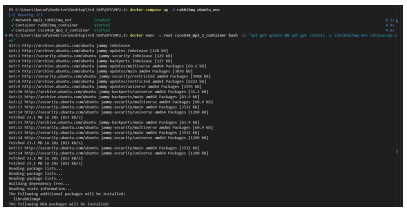
Objective: Verify that FOLLOW operations propagate across all clusters and that TIMELINE streaming correctly displays posts from every user with accurate poster IDs and no duplication.

Procedure:

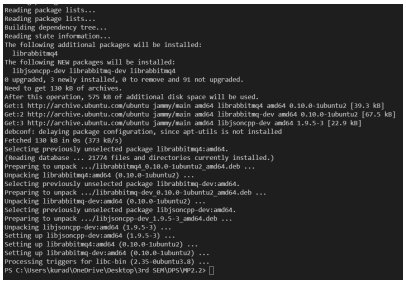
1. Login Users 1, 2, and 3.
2. Each user issues FOLLOW on the other two users, forming a fully connected follower graph.
3. All users enter `TIMELINE` mode.
4. Each user posts two messages (e.g., `p11`, `p12`, `p21`, `p22`, etc.).

Expected Result: All users' timelines include posts from all others across clusters. Poster IDs are displayed with each message. No duplicate posts appear and self-echo is suppressed.

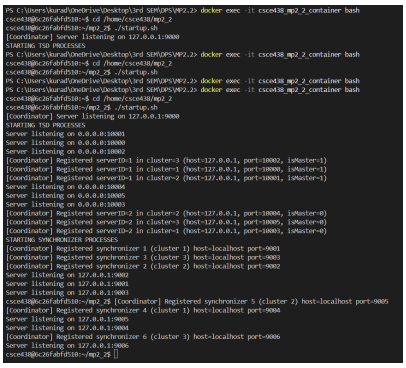
Screenshots for Test Case 2



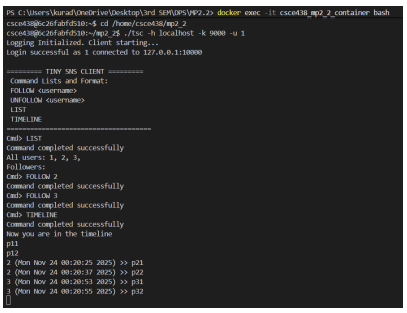
Staring env RabbitMQ and ubuntu



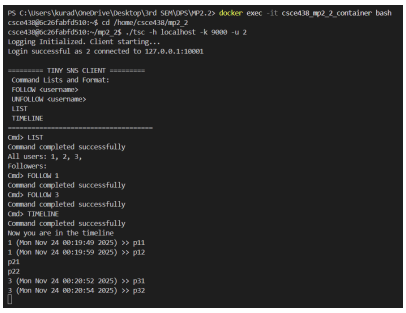
Starting containers



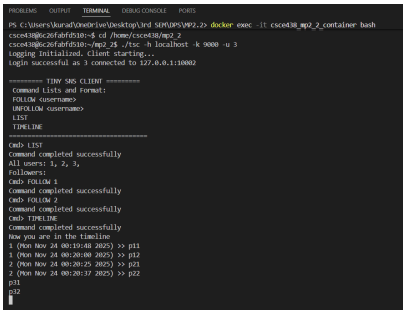
Starting Coordinator, Servers and Synchronizers



Creating User 1



Creating User 2



Creating User 3

Figure 3: Test Case 2 Execution Screenshots

5.3 Test Case 3: Adding a New User After Existing History

Objective: Validate that when a new user joins after previous users have already exchanged timeline history, the system correctly preserves old posts, maintains global user consistency across clusters, and ensures that new followers only receive posts from the follow point forward.

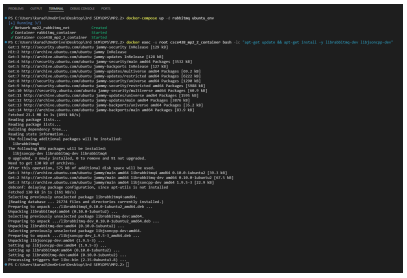
Procedure:

1. Start the full system and create Users 1, 2, and 3.
2. Establish FOLLOW relationships among Users 1, 2, and 3.
3. Each of Users 1, 2, and 3 posts multiple messages to generate initial history.
4. Kill all active client processes.
5. Restart the full system while preserving all on-disk cluster files.
6. Re-login Users 1, 2, and 3 (files already exist, no duplicates created).
7. Login new User 5.
8. Establish FOLLOW relationships:
    - User 5 follows Users 1, 2, and 3.
    - Users 1, 2, and 3 follow User 5.
9. User 5 posts new messages (e.g., `p51`, `p52`).

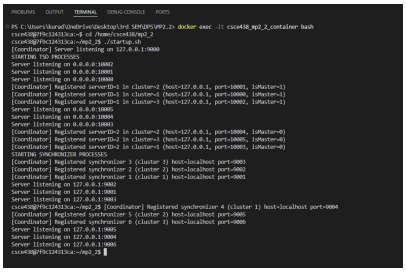
Expected Result:

- The global LIST command now shows {1, 2, 3, 5} across all clusters.
- Old timeline history among Users 1, 2, and 3 is preserved exactly as before restart.
- User 5 does **not** receive older posts from Users 1, 2, and 3 (no backfill).
- Users 1, 2, and 3 immediately receive User 5's new posts `p51`, `p52`.
- Follower and follow_list files correctly reflect the new bidirectional relationships in every cluster.
- No duplicate entries appear in `all_users.txt` or follower lists after re-login.

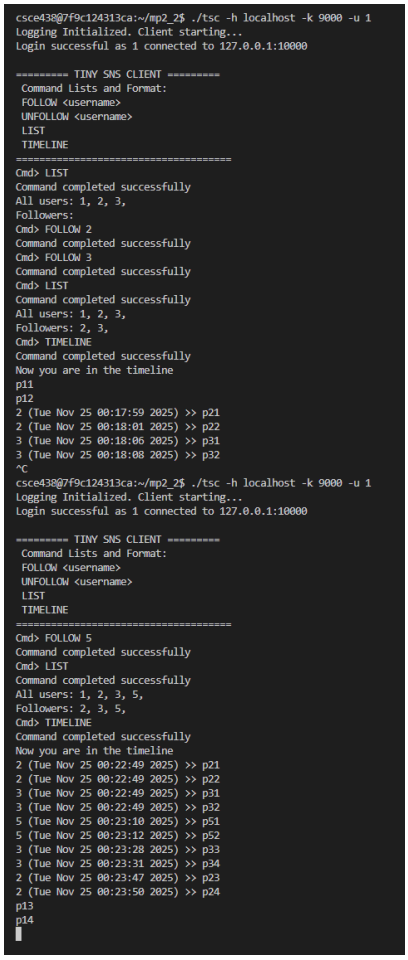
Screenshots for Test Case 3



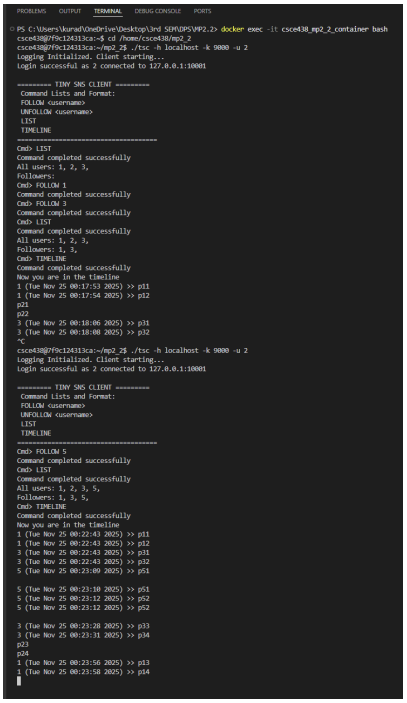
Staring env RabbitMQ and ubuntu



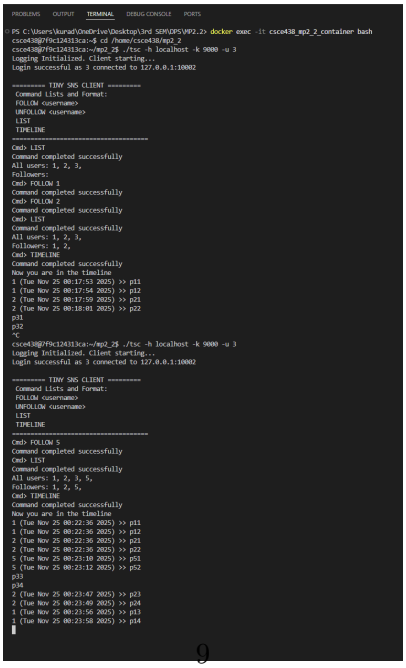
Starting Coordinator, Servers and Synchronizers



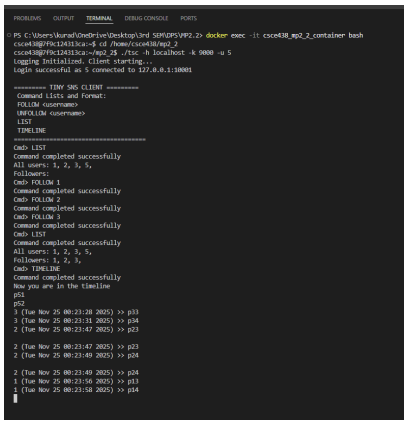
Creating User 1



Creating User 2



Creating User 3



Creating User 5

6 Notes and Assumptions

- Synchronizers provide eventual consistency, not strict ordering.
- No duplicate writes occur because synchronization logic deduplicates entries.
- Timeline streaming is best-effort real time.
- No backfill on new follows; timelines only grow forward.

Note: All the commands mentioned above were executed according to the Test Case Design Document.

7 Implementation Summary

7.1 Key Files

- **tsd.cc**: gRPC handlers, file persistence, streaming logic, follow/timeline fan-out.
- **synchronizer.cc**: Cross-cluster merges and RabbitMQ propagation.
- **coordinator.cc**: Registration, GetServer, heartbeats, discovery.
- **startup.sh**: Launches Coordinator, 6 TSDs, 6 Synchronizers.

7.2 Behavioral Highlights

- LIST always returns global union.
- FOLLOW/UNFOLLOW validated globally and replicated cluster-wide.
- TIMELINE streams latest posts with watchers.
- Posts stored once and propagated immediately to all followers.
- Synchronizer provides global merging and cross-cluster event propagation.

7.3 Synchronization Loop

- Merge all_users across clusters.
- Rebuild global follower lists from follow_list.
- Publish/consume timeline events from all synchronizers.
- Sleep 5 seconds per cycle.